A APPENDIX

A.1 Diagnosis Tool

These diagnosis tools leverage DSL programming to define complex abnormal scenarios. Existing scripting languages (e.g., Python) are utilized with standardized input-output formats, relying on monitoring platforms for metric and configuration data access. For example, for `LOG_FILE_SYNC`, `DBAIOps` includes two tools to enhance diagnosis accuracy and efficiency.

- *(1) LogSync Performance Verifier:* The tool automatically detects and analyzes database performance bottlenecks by monitoring key metrics (e.g., log file sync wait time, redo generation rate, transaction commits) against baseline threshold, identifying anomalies, and providing root cause insights (e.g., excessive commits, redo overload, undersized log buffers);
- *(2) RedoArchive Health Inspector:* The tool analyzes discrepancies between archive log and redo log sizes, evaluates parameter configurations (e.g., `log_buffer`, `archive_lag_target`), and identifies performance risks caused by `rapid_redo_log` switching or suboptimal settings, while providing technical rationales and compliance checks. The analysis includes verification of log file size and detection of any abnormal log patterns.

A.2 Prompt

`DBAIOps` instructs LLM to generate structured diagnosis reports, ensuring clear and actionable assessments of detected anomalies. The prompt template is designed to produce high-quality diagnoses that are both accurate and easy to understand.

[1. Anomaly Description]
- A concise problem statement or anomaly observation.
- Example: "CPU usage spiked to 95% at 16:00 on 2023-10-05."

[2. Alarm Rule Description]
- Define the alarm triggering conditions.
- Example: "Trigger if CPU usage exceeds 90% for more than 5 minutes."

[3. Metric Details]
- Key metrics involved in the analysis.
- Metric Name: CPU Usage (Percentage)
- Time Range: 1684600070 - 1684603670 (UNIX timestamps)
- Threshold: 90%

[4. Diagnosis Experience]
- Contextual information about the system/environment.
- The monitored server handles 10k requests/minute under normal conditions.
- Recent maintenance activities: Kernel update on 2023-10-04.

[5. Output Requirements]
- *Output format: Structured JSON format preferred*

Table 1: Report Generation Prompt Template in `DBAIOps`.

A.3 Diagnosis Report

We adopt Human Evaluation Accuracy (`HEval`) to measure the effectiveness of different methods, strictly adhering to three criteria. (1) Root Cause Recall: Whether identify all the relevant root causes; (2) Theoretical Consistency: Whether the diagnosis reasoning aligns with theoretical knowledge logically (e.g., adhering to Oracle database principles and operating system mechanisms); (3) Evidence Authenticity: Whether the evidence data (e.g., redo log write times, storage latency metrics) supporting the diagnosis reasoning is valid and free from hallucinations.

A.2.1 Oracle Diagnosis Report

The following two examples (Case 1 and Case 2) from the log synchronization delay scenario, where high LOG FILE SYNC wait time (10.73ms) triggered system performance risk alerts, will be used to illustrate how these three criteria serve as measurement standards for evaluating accuracy:

- Case 1 with high LOG FILE SYNC wait time (10.73ms) and system performance risk alert achieved 100% accuracy rate. (`DBAIOps` (DeepSeek-R1 671B))
- Case 2 with high LOG FILE SYNC wait time (10.73ms) and system performance risk alert achieved 40% accuracy rate.(DeepSeek-R1 32B)

Table 2: Case Study of Oracle Diagnosis Report (Positive Example).

Anomaly: LOG SYNCHRONIZATION DELAY

Abnormal Metrics List:

Metric ID	Metric Name	Min	Max	Avg
2184301	log file sync average wait time	1.08 ms	15.2 ms	6.0 ms
2184305	log file parallel write average wait time	0.78 ms	7.09 ms	3.0 ms
000014	Mem unavailable	72.82	73.07	73.0

Root Cause:

(1) Insufficient I/O Performance of REDO Log Storage:

- ***Evidence Chains:***
  Metric 2184301 (*log file sync*): max = 15.2 ms, avg = 6.0 ms (abnormal)
  Metric 2184305 (*log file parallel write*): max = 7.09 ms, avg = 3.0 ms (abnormal)
- ***Anomaly Analysis:***
  According to the formula: log_file_sync / log_file_parallel_write = 6.0 / 3.0 = 2
  Ratio in range (1.5, 2] $\Rightarrow$ Storage I/O performance is a primary factor

(2) Intermittent I/O Pressure Spikes During Log Writing:

- ***Evidence Chains:***
  At 06:00, log file sync spiked to 15.2 ms (vs. hourly avg 4.54 ms)
  log file parallel write peaked at 7.09 ms (high for mechanical disks)
  OS I/O latency = 0.47 ms (normal)
- ***Anomaly Analysis:***
  Storage likely under transient load

DBA Feedback:

The predefined hypothesis of the experiment is that the bottleneck lies in I/O storage. It can be observed that the two root causes identified in this case are both centered around this hypothesis, thus satisfying the **first evaluation criterion**. Furthermore, the evidence chains for the two root causes in this case are logically consistent and align with theoretical knowledge of Oracle and operating systems, satisfying the **second evaluation criterion**. Finally, the data cited in the evidence chains of this case are derived from actual experimental results, with no hallucinated content, satisfying the **third evaluation criterion**. Therefore, the accuracy of this case is **100%**.

A.2.2 PostgreSQL Diagnosis Report

The following three examples (Case 1 and Case 2) from the abnormal alarm on the metric < *backend buffer write ratio* > in PostgreSQL, which indicates potential misconfigurations in *shared buffers* or *bgwriter/checkpoint* settings, will be used to illustrate how these three evaluation criteria are applied to assess accuracy:

- Case 1 with backend buffer write ratio anomaly achieved a 100% accuracy rate. (`DBAIOps` (DeepSeek-R1 32B))
- Case 2 with backend buffer write ratio anomaly achieved a 0% accuracy rate. (DeepSeek-R1 32B)

A.2.3 DM8 Diagnosis Report

The following two examples (Case 1 and Case 2) from the sudden surge in REDO log volume scenario, where abnormal alarms were triggered in the database, will be used to illustrate how these three criteria serve as measurement standards for evaluating accuracy:

- Case 1 with sudden surge in REDO log volume and database abnormal alarm achieved 100% accuracy rate.(`DBAIOps` (DeepSeek-R1 32B))
- Case 2 with sudden surge in REDO log volume and database abnormal alarm achieved 40% accuracy rate.(DeepSeek-R1 671B)

Table 3: Case Study of Oracle Diagnosis Report (Negative Example).

Anomaly: LOG SYNCHRONIZATION DELAY

Abnormal Metrics List:

Metric ID	Metric Name	Min	Max	Avg
2184301	log file sync average wait time	0.75 ms	13.82 ms	5.0 ms
2184302	db file sequential read average wait time	0.08 ms	8.94 ms	1.0 ms
2184305	log file parallel write average wait time	1.36 ms	6.43 ms	3.0 ms
2189016	Redo Generated Per Sec	12.39	8485.84	220.03
000014	Mem unavailable	72.61	72.85	73.0

Root Cause:

(1) Log File Parallel Write Average Wait Time Anomaly:

- ***Evidence Chains:***
  Metric ID: 2184305 (*log file parallel write average wait time*): min = 1.36 ms, max = 6.43 ms, avg = 3.0 ms (abnormal).
- ***Anomaly Analysis:***
  The LGWR process's write latency directly affects the Log File Sync wait time during transaction commits. When LGWR's wait time increases, transactions need to wait longer to complete log writing. This increases Log File Sync latency.

(2) Redo Generation Rate Anomaly:

- ***Evidence Chains:***
  Metric ID: 2189016 (*redo generated per second*): min = 12.398, max = 485,838.96, avg = 44,220 (abnormal).
- ***Anomaly Analysis:***
  A sudden surge in redo log generation rate may exceed LGWR's write capability. This leads to a log buffer overflow. This requires waiting for LGWR to clear the buffer. This significantly increases the wait time for Log File Sync.

(3) Checkpoint Delay Anomaly:

- ***Evidence Chains:***
  Metric ID: 2180503 (*checkpoint delay*): min = 60,460 ms, max = 61,660 ms, avg = 61,060 ms (normal).
- ***Anomaly Analysis:***
  Excessive checkpoint delay (over 60 seconds) may prevent dirty data from being written to disk in a timely manner. This adds to LGWR's write load. This indirectly affects Log File Sync performance.

(4) Insufficient Memory:

- ***Evidence Chains:***
  Metric ID: 3000014 (*Mem unavailable*): min = 72.61, max = 72.85, avg = 73.0 (abnormal).
- ***Anomaly Analysis:***
  Memory pressure may cause the log buffer space to be insufficient. This frequently triggers forced flushes by LGWR. This extends the Log File Sync wait time.

(5) Control File Write Performance Anomaly:

- ***Evidence Chains:***
  Metric ID: 2184306 (*control file parallel write average wait time*): min = 0.59 ms, max = 3.78 ms, avg = 1.0 ms (abnormal).
- ***Anomaly Analysis:***
  Control file write delays could be caused by I/O subsystem bottlenecks or configuration issues. This further impacts LGWR's overall performance.

DBA Feedback:

The five root causes do not directly point to an I/O storage bottleneck as the direct cause of the alarm. Root causes (1), (3), and (5) are manifestations of the I/O storage bottleneck, indicating insufficient depth in analysis. Root causes (2) and (4) are correlated with the abnormal metrics, but they are not the direct cause of the alarm. Therefore, the **first evaluation criterion** is not fully satisfied, but the analysis is close to the cause, allowing for a certain degree of accuracy in the evaluation. Looking at the logical chains of the five conclusions, they mostly align with the basic theories of Oracle and operating systems, satisfying the **second evaluation criterion**. Finally, reviewing the evidence chains of the five conclusions, in conclusions (3) and (5), the data cited in the evidence chains do not exist in the provided metrics, indicating hallucinations by the large model. This completely fails to meet the criterion, resulting in a significant hallucination issue. The **first evaluation criterion** has a certain degree of accuracy, the **second evaluation criterion** is met, and the **third evaluation criterion** is completely unmet. Therefore, the accuracy of the analysis of this case is assessed at **40%**.

Table 4: Case Study of PostgreSQL Diagnosis Report (Positive Example).

Anomaly: BACKEND PROCESS FLUSHES DIRTY PAGES IN LARGE QUANTITIES

Abnormal Metrics List:

Metric ID	Metric Name	Min	Max	Avg
2300102	Active Sessions	1.0	52.0	25.0
2300111	Non-backend Process Shared Cache Write Ratio	11.0	100.0	63.0
2300114	Physical Reads Per Second	0.0	2496.66	1017.0
2300115	Transactions Per Second	1.63	127.74	53.0
2300116	Commits Per Second	1.58	116.45	49.0
2300122	Inserts Per Second	0.0	721.26	297.0
2300123	Updates Per Second	0.0	1370.27	566.0
2300133	Checkpoint Write Sync Time	0.0	163.17	25.0
2300135	Shared Cache Writes Per Second	0.0	1071.62	436.0
2300137	Bgwriter Process Shared Cache Writes Per Second	0.0	297.46	126.0
2300138	Backend Process Shared Cache Writes Per Second	0.0	856.01	307.0
2300139	Log Generation Per Second	0.73	4339516.47	1459389.0
2300140	Bgwriter Process Stop Scan Count Per Second	0.0	2.76	1.0
2300152	Log Writes Per Second	0.73	4339516.47	1459389.0
2300196	Bgwriter Process Shared Cache Write Ratio	0.0	50.0	17.0
2300197	Backend Process Shared Cache Write Ratio	0.0	89.0	37.0
3000006	I/O Latency	0.0	5736.96	827.0
3000100	I/OPS	0.0	873.0	225.0
3000101	I/O KBPS	0.0	13544.0	3020.0

Root Cause:

(1) Surge in Backend Process Write Due to bgwriter_lru_maxpages Parameter Setting Too Low:

- ***Evidence Chains:***

  Metric 2300140 (*Bgwriter Stop Scan Count Per Second*) showed an average of 1.0, with a maximum of 2.76 and a minimum of 0.

- ***Anomaly Analysis:***

  According to background knowledge, when the stop scan count is greater than 0, it indicates that the bgwriter process has reached the `bgwriter_lru_maxpages` limit and stopped. The current value of the parameter is set to 100, and frequent stop scan occurrences prevent the bgwriter from efficiently processing dirty pages, forcing the backend process to take over the dirty page write operations.

(2) Surge in I/O Latency Causing Inadequate Storage I/O Performance:

- ***Evidence Chains:***

  Metric 3000006 (*I/O Latency*) showed an average of 827ms, with a peak of 5736ms, indicating abnormal behavior.

- ***Anomaly Analysis:***

  Excessive disk latency caused bgwriter and checkpoint processes to fail to complete their write operations on time, forcing backend processes to take over the dirty page flushing operations.

(3) Surge in Backend Process Write Due to bgwriter_lru_multiplier Parameter Set Too Low:

- ***Evidence Chains:***

  Metric 2300196 (*Bgwriter Cache Write Ratio*) showed that only 17% of dirty pages were handled by the bgwriter on average, far below the reasonable threshold.

- ***Anomaly Analysis:***

  According to background knowledge, the `bgwriter_lru_multiplier` controls the amount of dirty pages to be flushed during each cycle. If this parameter is set too low, even with sufficient `bgwriter_lru_maxpages`, it can lead to inefficient dirty page flushing.

DBA Feedback:

The predefined hypothesis of the experiment is that the system is facing BGWRITER parameter issues and I/O bottlenecks. It can be observed that the three root causes identified in this case are primarily centered around the predefined root causes. The first root cause analyzes the issue with bgwriter parameter settings, while the second one focuses on I/O bottlenecks. Thus, the **first evaluation criterion** is fully satisfied. Furthermore, the evidence chains for both root causes in this case are logically clear, aligned with Oracle and operating system theoretical knowledge, satisfying the **second evaluation criterion**. Finally, the data referenced in the evidence chains of this case comes directly from the actual experimental data, with no hallucinated or fabricated content, satisfying the **third evaluation criterion**. Therefore, the accuracy of this case is **100%**.

Table 5: Case Study of PostgreSQL Diagnosis Report (Negative Example).

Anomaly: BACKEND PROCESS FLUSHES DIRTY PAGES IN LARGE QUANTITIES

Abnormal Metrics List:

Metric ID	Metric Name	Min	Max	Avg
2300102	ActiveSessions	26.0	55.0	48.0
2300114	PhysicalReadsPerSec	636.88	3537.25	1248.0
2300115	TransactionsPerSec	29.24	151.05	85.0
2300116	CommitsPerSec	26.32	146.03	81.0
2300122	InsertRowsPerSec	142.71	391.03	233.0
2300123	UpdateRowsPerSec	287.38	702.78	442.0
2300133	CheckpointWriteSyncTimePerSec	0.0	208.5	74.0
2300135	SharedBufferWritePerSec	192.18	750.04	426.0
2300139	LogGenerationPerSec	835479.27	2484808.74	1500813.0
2300140	BgWriterProcessStopScanCountPerSec	1.22	2.24	2.0
2300145	CheckpointDelay	12848952.0	525058688.0	204486275.0
2300152	LogWritePerSec	835479.27	2484808.74	1500813.0
2300153	ConnectionUsageRate	60.0	64.0	62.0
2300196	BgWriterProcessSharedBufferWriteRatio	26.0	75.0	54.0
2300197	BackendProcessSharedBufferWriteRatio	21.0	73.0	45.0
3000006	I/O Latency	969.19	10923.3	3942.0
3000100	I/O PS	138.0	716.0	332.0
3000101	I/O KBPS	1192.0	5936.0	2824.0

Root Cause:

(1) High I/O Latency Leading to Inefficient BgWriter and Checkpoint Processes:

- ***Evidence Chains:***
  Metric ID 3000006 (*I/O Latency*) had a maximum value of 10,923.3ms and an average of 3,942ms (normal value should be <10ms).
  Metric ID 2300135 (*SharedBufferWritePerSec*) had a maximum value of 750.04 (abnormally high write pressure).
  Metric ID 2300144 (*CheckpointDelay*) had a maximum value of 525,058,688.0 (checkpoint process severely blocked).

(2) Misconfigured Checkpoints Leading to Dirty Page Accumulation:

- ***Evidence Chains:***
  Metric ID 2300144 (*CheckpointDelay*) had an average of 204,486,275 (abnormal when >100MB).
  Metric ID 2300136 (*CheckpointWritePerSec*) had an average of only 4.0 (indicating ineffective dirty page cleanup by checkpoint).
  Metric ID 2300135 (*SharedBufferWritePerSec*) consistently showed high values.

(3) Insufficient Configuration of BgWriter Process:

- ***Evidence Chains:***
  Metric ID 2300196 (*BgWriterWriteRatio*) had a minimum value of 26.0% (far below the normal range).
  Metric ID 2300140 (*BgWriterStopScanCountPerSec*) had an average of 2.0 times per second (frequent scan stoppages).
  Metric ID 2300137 (*BgWriterWritePerSec*) had an average of 207.0 (much lower than backend's 215.0).

(4) High Concurrent Write Operations Exceeding System Processing Capacity:

- ***Evidence Chains:***
  Metric ID 2300102 (*ActiveSessions*) had an average of 48 (indicating high concurrency).
  Metric ID 2300123 (*UpdateRowsPerSec*) had an average of 442 (sustained high write pressure).
  Metric ID 2300139 (*WALGenerationPerSec*) had an average of 1.5MB/s (indicating large-scale writes).

(5) Unoptimized SQL Leading to Additional Write Pressure:

- ***Evidence Chains:***
  Metric ID 2300122 (*InsertRowsPerSec*) had an average of 233.
  Metric ID 2300123 (*UpdateRowsPerSec*) had an average of 442.
  Metric ID 3882 (*FullTableScanPercentage* > 90%) was triggered.

DBA Feedback:

Among the five root causes, (1) and (3) indicate high concurrency, and (2) reflects high system load—these three are largely valid. However, root causes (4) and (5) mainly describe transaction backlog, which is not directly related to the volume of REDO logs. REDO generation is primarily influenced by the number of modification operations in transactions. While the correct root causes are partially covered, two incorrect ones are also present. Therefore, the **first evaluation criterion** is assessed at **0%**. Examining the logical chains of the five root causes reveals that they are mostly derived from co-occurring alarms. The reasoning lacks rigor and fails to construct a complete logical chain based on database principles. As such, the **second evaluation criterion** is **not satisfied**. Finally, regarding the evidence chains of the five root causes, the above case mainly uses real-time alarm data provided in the dataset. The evidence is genuine and aligns with the data, satisfying the **third evaluation criterion**.[18] The **first evaluation criterion** is **not met** (0%), the **second evaluation criterion** is **not met**, and the **third evaluation criterion** is **met**. Therefore, the overall accuracy of this case is assessed at **0%**.

Table 6: Case Study of DM8 Diagnosis Report (Positive Example).

Anomaly: REDO LOG VOLUME EXCESSIVE

Abnormal Metrics List:

Metric ID	Metric Name	Min	Max	Avg
2250001	ExecutionsPerSec	2.48	2267.77	1203.0
2250004	PhysicalReadsPerSec	0.0	202.63	64.0
2250005	RedoGeneratedPerSec	21.02	842271.74	460098.0
2250009	UserTransactionPerSec	0.06	123.23	65.0
2250010	NetworkTrafficVolumePerSec	33130.0	111231363.0	57730120.0
2250025	DataPageChanges	0.02	4113.39	2192.0
2250041	DatabaseTimePerSec	4.75	14321.87	10691.0
2250043	AverageActiveSessions	1.0	16.0	12.0
3000003	CPUused	8.26	55.23	39.0
3000013	BlockedProcess	2.0	19.0	12.0

Root Cause:

(1) Surge in Data Page Changes Due to High-Frequency DML Operations:

- ***Evidence Chains:***

Metric 2250025 (*data page changes*) had an average value of 2,192 during the alert period, which is hundreds of times higher than the baseline, directly triggering REDO log generation.

- ***Anomaly Analysis:***

The most time-consuming SQL statements included frequent UPDATE operations (e.g., SQL_ID 641), with single execution time exceeding 13 seconds. These involved batch updates with range conditions (e.g., `ol_o_id`), causing massive data page modifications.

(2) Abnormally High Transaction Commit Frequency:

- ***Evidence Chains:***

Metric 2250009 (*UserTransactionPerSec*) showed an average of 65 and a peak of 123.23, far exceeding normal levels, corroborated by alerts such as "transactions per second exceeded 10x the hourly average". Metric 2250001 (*ExecutionsPerSec*) also showed an elevated average of 1,203.

- ***Anomaly Analysis:***

SQL diagnosis indicated FOR UPDATE lock contention (e.g., SQL_ID 618), which delayed transaction commits and indirectly increased REDO log segment writes.

(3) Inadequate RLOG_POOL_SIZE Configuration:

- ***Evidence Chains:***

The configured `RLOG_POOL_SIZE` was 256MB, but based on workload, the estimated minimum required was 626.21 MB.

- ***Anomaly Analysis:***

Actual REDO generation peaked at 842,271.74 bytes/sec (≈ 822 KB/sec), while the insufficient log buffer throughput caused frequent log flushing. This increased pressure on the I/O subsystem, consistent with the alert "I/O wait time suddenly spiked".

DBA Feedback:

The predefined hypothesis of the experiment is that the system is under high load and high concurrency. It can be observed that the first two root causes identified in this case are both centered around this hypothesis. In addition, the third root cause analyzes the insufficient RLOG_POOL_SIZE configuration under high-load conditions, which demonstrates the model's ability to generalize and identify configuration flaws. Thus, the **first evaluation criterion** is fully satisfied. Furthermore, the evidence chains for all three root causes in this case are logically sound and aligned with the theoretical framework of DM database systems, satisfying the **second evaluation criterion**. Finally, the data cited in the evidence chains of this case are all derived from the actual experiment, with no hallucinated or fabricated content, satisfying the **third evaluation criterion**. Therefore, the accuracy of this case is **100%**.